Prática de Laboratório 02

Disciplina: Análise de Algoritmos – Prof. Pedro Ximenes

Semestre 2026.2 – UNICAP

Instruções Gerais

- Formem duplas.
- Vocês podem utilizar a IDE instalada nas máquinas do laboratório ou o compilador online: `https://www.online-python.com/`.
- Executem os códigos fornecidos, preencham as tabelas com os valores medidos e respondam às perguntas propostas.

---

1 Missão 1: Teste de Primalidade ($O(n)$ vs $O(\sqrt{n})$)

O objetivo desta missão é comparar duas abordagens para verificar se um número inteiro positivo n é primo: o teste ingênuo, que percorre todos os divisores potenciais de 2 até $n-1$, e o teste otimizado, que encerra as verificações em $\sqrt{n}$.

Passo 1: Abram o arquivo `codigo1.py` (ou copiem o trecho abaixo) e executem o programa.

```python
import time

def primo_linear(n):
    passos = 0
    if n <= 1: return False, passos
    for i in range(2, n):
        passos += 1
        if n % i == 0: return False, passos
    return True, passos

def primo_raiz(n):
    passos = 0
    if n <= 1: return False, passos
    i = 2
    while i * i <= n:
        passos += 1
        if n % i == 0: return False, passos
        i += 1
    return True, passos
```

Preenchimento e Análise (Anotem no relatório)

1. Por que foram escolhidos números primos para o teste de desempenho? O que ocorreria com o número de passos se n fosse um número par grande como 10.000.000?

Valor de n	Passos $O(n)$	Tempo $O(n)$	Passos $O(\sqrt{n})$	Tempo $O(\sqrt{n})$
100.003	_____	_____	_____	_____
1.000.033	_____	_____	_____	_____
10.000.019	_____	_____	_____	_____

2. Observe o número de iterações do algoritmo `primo_raiz` para $n = 10.000.019$. Calcule $\sqrt{10.000.019}$. Qual é a relação entre esse valor e a quantidade de passos medida?

3. Por que a condição de parada $i \times i \leq n$ (ou $i \leq \sqrt{n}$) é matematicamente suficiente para determinar a primalidade de qualquer inteiro positivo n?

---

2 Missão 2: Algoritmos de Ordenação (Insertion Sort vs Selection Sort)

Ambos os algoritmos possuem complexidade de pior caso $O(n^2)$. Contudo, eles apresentam comportamentos distintos conforme a organização inicial dos dados de entrada.

Passo 1: Abram e executem o arquivo `codigo2.py`. Ele contabiliza duas operações fundamentais: **comparações entre elementos** e **trocas/deslocamentos** para instâncias de tamanho $N = 2.000$.

Preenchimento e Análise (Anotem no relatório)

	Insertion Sort		Selection Sort	
Cenário ($N = 2.000$)	**Comparações**	**Trocas/Movs**	**Comparações**	**Trocas/Movs**
1. Já Ordenado	_____	_____	_____	_____
2. Invertido	_____	_____	_____	_____
3. Aleatório	_____	_____	_____	_____

1. Observe o número de comparações do **Selection Sort** nos três cenários. Por que essa contagem é sempre exatamente igual a $\frac{N(N-1)}{2}$, independentemente da ordenação prévia do vetor?

2. Analise o comportamento do **Insertion Sort** sobre a entrada já ordenada. Qual foi a complexidade assintótica observada e qual mecanismo do algoritmo justifica esse resultado?

3. Em um cenário onde a operação de escrita em memória (troca) possui custo significativamente maior do que a leitura (comparação), qual dos dois algoritmos é mais eficiente ao ordenar um vetor inversamente ordenado? Justifique utilizando os dados medidos na tabela.

---

3 Missão 3: Inserção no Início (Vetor Dinâmico vs Lista Encadeada)

Em arranjos contíguos (vetores), inserir um elemento no índice 0 exige deslocar todos os elementos já existentes uma posição à direita. Em listas encadeadas, a inserção no início envolve apenas a criação do nó e a atualização do ponteiro de início (*head*).

Passo 1: Executem o arquivo `codigo3.py`. O código mede o tempo para realizar N inserções sucessivas no início usando a lista padrão do Python (`list.insert(0, x)`) e uma lista simplesmente encadeada (`ListaEncadeada`).

Preenchimento e Análise (Anotem no relatório)

Quantidade de Inserções (N)	**Tempo Vetor (`list.insert(0)`)**	**Tempo Lista Encadeada**
10.000	_____	_____
30.000	_____	_____
60.000	_____	_____

1. Ao aumentar N de 10.000 para 60.000 (um fator de 6):
   - Por qual fator o tempo da **Lista Encadeada** aumentou?
   - Por qual fator o tempo do **Vetor** aumentou?
2. Demonstre matematicamente por que N inserções consecutivas na primeira posição de um vetor de tamanho variável resultam em um custo acumulado $\Theta(N^2)$, enquanto na lista encadeada o custo é $\Theta(N)$.
3. Considere uma lista simplesmente encadeada que armazene ponteiros para o primeiro (`head`) e para o último nó (`tail`). Qual é a complexidade assintótica da operação de remover o último elemento da lista? Justifique.

---

4 Missão 4: Algoritmos Recursivos e Teorema Mestre

A abordagem de divisão e conquista decompõe uma instância em subproblemas menores. A relação de recorrência $T(n) = a\,T(n/b) + f(n)$ expressa o custo total das chamadas recursivas e da etapa de combinação.

Passo 1: Executem o arquivo `codigo4.py`. O programa avalia duas estruturas de divisão recursiva:

- **Busca Binária:** $T(n) = T(n/2) + O(1) \implies a = 1, b = 2.$
- **Divisão Dupla:** $T(n) = 2\,T(n/2) + O(1) \implies a = 2, b = 2.$

Tamanho n	Busca Binária ($a = 1$)		Divisão Dupla ($a = 2$)	
	Profundidade	**Total Chamadas**	**Profundidade**	**Total Chamadas**
1.024	_____	_____	_____	_____
32.768	_____	_____	_____	_____
1.048.576	_____	_____	_____	_____

Preenchimento e Análise (Anotem no relatório)

1. Compare a profundidade máxima da pilha de chamadas entre os dois algoritmos para $n = 1.048.576$. Por que esse valor é idêntico em ambos?

2. Por que, mesmo apresentando a mesma profundidade de pilha (≈ 21), o número total de chamadas da Divisão Dupla ultrapassa 2 milhões, enquanto a Busca Binária realiza apenas 21 chamadas?

3. Utilizando o Teorema Mestre ($T(n) = a\,T(n/b) + \Theta(n^d)$), determine formalmente a classe de complexidade assintótica de cada um dos dois algoritmos analisados.